Labreport Advanced Computer Architecture
Lab 3

Jonas Roels & Niels Van Bossche

March 3, 2024



Class: MEIsw

Contents

1 Introduction **1**

2 Warping **1**

3 Divergence **2**

1 Introduction

In this lab we will develop a program to edit the colors of an image using the GPU, keeping in mind thread warping and thread divergence. It is important to note that all tests have been run on an Nvidia GTX 1060 6GB GPU with 1280 Cuda cores divided up in 10 stream multiprocessors [1]. The source code for this project can be found on `https://github.com/RoelsJonas/Advanced-Computer-Archticture-Lab-3`

2 Warping

For the first part of this lab a GPU function was developed where each core inverts a single color value (so either R,G or B) of a certain pixel. This core then strides further until it has reached the end of the data.
This function was run with a different amount of blocks (1, 2, 4, 8, 16 and 32) and for each of those a different amount of threads per block (16 up to 1024). The processing time for each of these runs was measured and can be found in Figure 1 (note the logarithmic scale on the y-axes). We can thus conclude that increasing the amount of blocks and the amount of threads in each block decreases the processing time. Without seemingly any large inefficiencies due to warping when only a few cores from a certain warp are used. Interestingly both the curves of the 16 and 32 blocks runs show a significant increase in processing time when exceeding 800 threads per block. We believe this is due to a scheduling inefficiency.

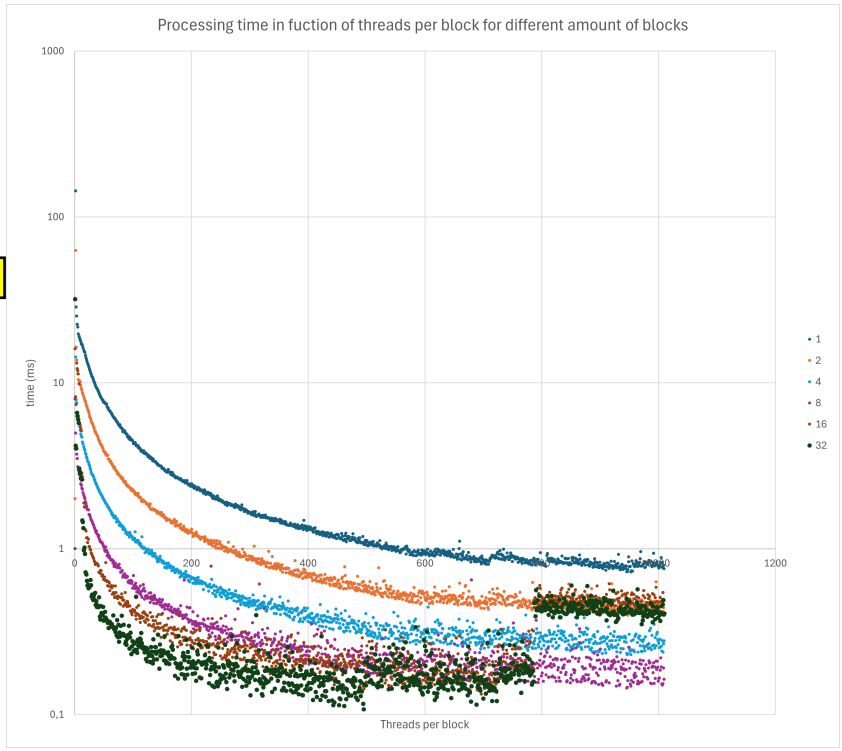


Figure 1: Comparison of the processing time for different number of blocks and threads per block.

3 Divergence

In the second part of the lab assignment, we retained the GPU function where each core inverts a single color value, focusing this time on colors G and B. However, for red-colored values, we introduced a function that requires longer processing time relative to blue and green pixels due to the modulo operation.
Similar to the previous section on Warping, we executed this function using varying numbers of blocks and threads per block, ranging from 16 to 1024. Consistent with our earlier observations, we found that increasing the number of blocks and threads per block led to decreased processing time.
Upon analyzing the graph, it was clear that some saturation occurred, particularly with configurations involving 16 and 32 blocks and a higher number of threads per block. This saturation suggests that the GPU reaches its limit in processing warps simultaneously. Additionally, an increase in scatter was observed with a higher number of blocks, this due to the fact that some configuration perform better and some worse thanks to the thread divergence. The greater the number of blocks and threads, the more possibilities for divergent execution paths, leading to increased scatter in processing time.

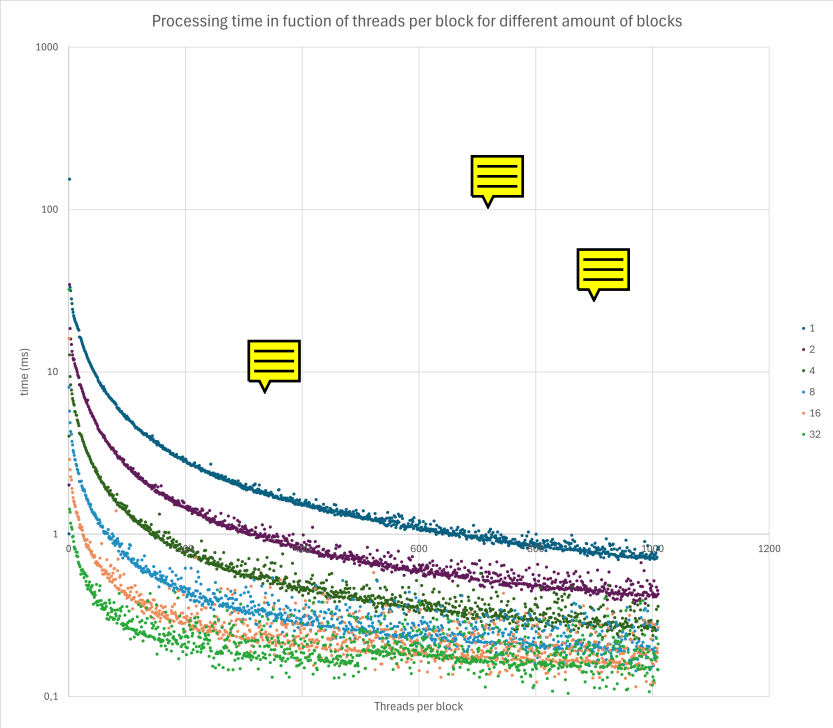


Figure 2: Comparison of the processing time required with different amount of blocks and threads per block for part 2.

References

[1] GeForce GTX 1060 out now. GTX 980-class performance for just $249. [Online]. Available: https://www.nvidia.com/en-us/geforce/news/gfecnt/nvidia-geforce-gtx-1060/